

Mini Job Board: Technology Framework Stack

This document summarizes the technological framework stack, external dependencies, security libraries, and testing components of the Mini Job Board project.

Technology Stack Overview

Component	Technology	Version	Description
Backend	Java SDK	17	Core language platform
Framework	Spring Boot	3.5.0	Web starter, Security, Data JPA, Validation
Database	MySQL	8.0+	Relational Database Management System
Frontend	React	19.2.4	Single Page Application framework
Routing	React Router DOM	7.14.0	Client-side page navigation
Build Tool	Vite	8.0.1	Next-generation frontend bundler
CSS Styling	Vanilla CSS	CSS3	Custom Glassmorphism layout aesthetics

Backend Technologies & Libraries

The backend is constructed using Spring Boot 3, leveraging several critical components.

1. Spring Security & JWT Authentication

- **Security Config:** `SecurityConfig.java` defines the security filter chain, disables CSRF for a stateless context, and enforces authentication rules.
- **BCrypt Hashing:** Raw passwords are encoded using salt generation before database storage to protect user credentials.
- **JWT (JSON Web Token):** Uses version `0.12.5`. The `JwtUtils` utility class handles token generation and parsing using HMAC keys from a 256-bit hexadecimal secret key.

2. Spring Data JPA & Hibernate 6

- **Database Connectivity:** Utilizes `mysql-connector-j` as the JDBC driver.
- **Hibernate ORM:** Configured via `application-dev.properties` to connect to the local MySQL instance (`jdbc:mysql://localhost:3306/minijobboard`). It uses `org.hibernate.dialect.MySQLDialect` for query translation.
- **JPA Auditing & Mapping:** Manages entity relationships, including `@OneToMany` cascades and `@ManyToMany` join tables. Custom native SQL queries are utilized to prevent N+1 query bottlenecks.



Frontend UI Technologies

The frontend is built using **React 19** and structured as a component-driven Single Page Application (SPA).

1. State Management (AuthContext)

- **Context API:** Used to expose global session variables, including the authenticated user, login state, and logout commands.
- **Lazy Initializer:** Component state is initialized by reading `localStorage` synchronously before the component mounts to prevent layout flashing.
- **Persistent Storage:** Username changes on the profile page are synchronized directly into local storage to ensure consistent navigation displays.

2. Styling System

- **Vanilla CSS:** Styles are managed centrally within `src/index.css`.
- **Aesthetics:** Glassmorphism attributes, such as transparency, `backdrop-filter: blur()`, rounded corners, and soft gradients, are used for a modern interface.



Testing and Quality Assurance

Unit testing is designed around isolation principles to ensure fast local execution.

Core Testing Tools

- **JUnit 5 (Jupiter):** Orchestrates the test runner framework, providing annotations like `@Test` and `@BeforeEach`.
- **Mockito:** Extends testing using `@ExtendWith(MockitoExtension.class)` to mock repository interfaces (e.g., `JobRepository`, `UserRepository`), bypassing actual database writes.
- **Security Stubbing:** Mocks the `SecurityContextHolder` to inject fake authenticated users during service unit execution tests.